Seminario de Arquitecturas de Software: Un Análisis Comparativo de Estilos Basados en Capas y Dominio

Segunda entrega: Arquitecturas de Software Distribuidas

Ingeniería de Software

Equipo: HAVING FE

Integrante	**Pregunta(s)**
Eric Reyes Milián	Pregunta 1
Adrián Estévez Álvarez	Pregunta 2
Alfonso Tejeda Rodríguez	Pregunta 3
Kamila Reinoso Asín	Pregunta 4
Miguel Andrés Cazorla Zamora	Pregunta 5
Reynaldo Guillermo Capdevila Reyes	Pregunta 6

Repositorio: https://github.com/alfonsotejeda/astarship

12 de noviembre de 2026

Índice

Enunciados de las preguntas **2**

1. Pregunta 1: Comunicación y acoplamiento **4**
1.1. Marco conceptual: connascencia estática y dinámica 4
1.2. Orquestación en microservicios 4
1.3. Coreografía en microservicios 5
1.4. Servicio de grano grueso en Service-Based Architecture 5
1.5. Fallo del pago después de notificar al ganador 5
1.6. Síntesis 6

2. Pregunta 2: Elasticidad extrema **7**
2.1. Por qué el cuello de botella es la base de datos 7
2.2. Por qué Space-Based Architecture resuelve el escenario 7
2.3. Colisiones de datos en cachés replicadas 8
2.4. Mecanismos de mitigación y trade-offs 9
2.5. Conclusión 9

3. Pregunta 3: Fallos y resiliencia en EDA **10**
3.1. Impacto del mensaje defectuoso 10
3.2. Solución: Workflow Event Pattern 11
3.3. Trade-offs que introduce 11

4. Pregunta 4: Granularidad y acoplamiento **12**
4.1. El error fundamental: confundir entidad con bounded context 12
4.2. Nueva estrategia de descomposición 13
4.3. Propuesta concreta 14
4.4. La operación de ńcontrataciónż 14

5. Pregunta 5: Atributos de calidad y trade-offs **15**
5.1. Requisito 1: consistencia de datos absoluta (ACID) 15
5.2. Requisito 2: despliegue independiente 16
5.3. Requisito 3: equipo de experiencia mixta 16
5.4. Síntesis y trade-offs que el CTO acepta 17

6. Pregunta 6: El legado de SOA **18**
6.1. La reutilización como fuente de acoplamiento 18
6.2. La inversión del principio en microservicios 18
6.3. Cuándo sigue siendo apropiada la orquestación 19

Referencias **21**

Declaración de Uso de la Inteligencia Artificial **22**

Enunciados de las preguntas

1. **Análisis Comparativo de Comunicación y Acoplamiento:** Un sistema de subastas en línea requiere que, al finalizar una subasta, se realicen múltiples acciones: notificar al ganador, procesar el pago y actualizar el inventario. En un diseño con **Microservicios**, un arquitecto podría optar por una coreografía de eventos (comunicación asíncrona) o una orquestación mediante un servicio mediador. En un diseño con **Service-Based Architecture**, estas acciones probablemente se ejecutarían dentro de un mismo servicio de dominio de grano grueso. Basado en los conceptos de *connascence* (connascencia) y los trade-offs de la comunicación síncrona vs. asíncrona, analice:

   - £Cómo afecta cada enfoque (orquestación vs. coreografía en microservicios, y servicios de grano grueso en SBA) al acoplamiento dinámico entre los componentes del sistema?
   - £Qué implicaciones tiene cada diseño para la tolerancia a fallos y la consistencia de los datos (ACID vs. BASE) si el servicio de pago falla justo después de que se notifica al ganador?

2. **Diseño para Elasticidad Extrema y Consistencia de Datos:** Considere el sistema de venta de entradas para conciertos descrito en el libro (Capítulo 15). Durante el lanzamiento de entradas para un artista popular, el sistema experimenta un aumento repentino de usuarios de cientos a decenas de miles en segundos. Se le pide como arquitecto diseñar la solución.

   - Justifique por qué una **Space-Based Architecture** sería una elección más adecuada que una **Microservices Architecture** para este escenario específico, enfocándose en las características de elasticidad y escalabilidad.
   - Sin embargo, la Space-Based Architecture introduce el problema de las colisiones de datos en cachés replicadas. Describa cómo un arquitecto podría mitigar el riesgo de colisiones de datos en el contexto de la venta de asientos numerados, donde la consistencia es crucial. £Qué mecanismos o trade-offs consideraría?

3. **Análisis de Fallos y Resiliencia en Arquitecturas Dirigidas por Eventos:** En una **Event-Driven Architecture** con topología de Broker, un procesador de eventos (*Event Processor*) falla repetidamente al procesar un tipo específico de mensaje debido a un error en los datos.

   - Describa el impacto de este fallo en el flujo general del sistema y en la experiencia del usuario final, considerando que no hay un mediador central.
   - Proponga una solución arquitectónica utilizando el *Workflow Event Pattern* (Patrón de Eventos de Flujo de Trabajo) descrito en el Capítulo 14. Explique detalladamente cómo este patrón mitiga el problema y qué componentes adicionales (como el *workflow processor* y el *dashboard*) serían necesarios, así como los trade-offs que introduce (ej. procesamiento fuera de orden).

4. **El Dilema de la Granularidad y el Acoplamiento en Microservicios:** Un equipo de desarrollo está migrando un sistema monolítico de gestión de recursos humanos a Microservicios. Inicialmente, crean servicios muy pequeños para cada entidad

(Empleado, Departamento, Nómina, Beneficios). Pronto descubren que completar una operación simple, como contratar a un nuevo empleado, requiere una compleja coreografía de llamadas síncronas entre casi todos los servicios, lo que resulta en un alto acoplamiento y una fragilidad similar a la del monolito original.

- Basado en los principios de *bounded context* y las guías de granularidad del Capítulo 17, £cuál fue el error fundamental en el diseño de los servicios?
- Proponga una nueva estrategia de descomposición. £Cómo aplicaría los conceptos de propósito, transacciones y coreografía para redefinir los límites de los servicios y reducir el acoplamiento? Proporcione un ejemplo de cómo agruparía los servicios para la operación de ńcontrataciónż.

5. **Comparación de Atributos de Calidad y Trade-offs:** Un director de tecnología (CTO) le pide que elija un estilo arquitectónico para un nuevo sistema de procesamiento de transacciones financieras. Los requisitos clave son: 1) Consistencia de datos absoluta (ACID), 2) Capacidad de desplegar nuevas características de negocio de forma independiente y rápida, y 3) Un equipo de desarrollo con experiencia mixta.

   - Compare **Service-Based Architecture** y **Microservices Architecture** en función de estos tres requisitos. £Cuál de las dos arquitecturas es más adecuada para cumplir con el requisito de consistencia de datos y por qué?
   - £Cómo se ve afectada la capacidad de despliegue independiente (requisito 2) en cada estilo? £Qué trade-offs debe aceptar el CTO si prioriza la consistencia sobre la agilidad de despliegue?

6. **El Legado de SOA y su Influencia en Arquitecturas Modernas:** La **Service-Oriented Architecture** (SOA) con orquestación a menudo es criticada por su complejidad y por crear un cuello de botella central (el motor de orquestación). Sin embargo, algunos de sus principios, como la reutilización de servicios, siguen siendo relevantes. SOA también admite coreografía, no solo orquestación.

   - Analice cómo la **Microservices Architecture** aborda el problema de la reutilización de una manera fundamentalmente diferente. £Cómo maneja la reutilización de lógica de negocio (ej. validación de clientes) vs. la reutilización operativa (ej. logging, monitoreo)?
   - £En qué escenarios un arquitecto podría todavía considerar un enfoque de orquestación (como un mediador local en microservicios o un *orchestration engine* en SOA) a pesar de sus desventajas? Proporcione un ejemplo de un flujo de trabajo complejo que se beneficiaría de la orquestación explícita.

1. Pregunta 1: Análisis comparativo de comunicación y acoplamiento (subastas en línea)

El cierre de una subasta dispara tres acciones: notificar al ganador, procesar el pago y actualizar el inventario. La pregunta no es cuál de los tres diseños es ńmejorż, sino qué forma de acoplamiento introduce cada uno y qué ocurre cuando una de las tres acciones falla. Para responderlo con precisión conviene separar dos nociones que suelen confundirse: el acoplamiento estático, que describe cómo están conectados los componentes en tiempo de construcción y despliegue, y el acoplamiento dinámico, que describe cómo se comunican entre sí en tiempo de ejecución (Ford et al., 2021).

1.1. Marco conceptual: connascencia estática y dinámica

La connascencia, introducida por Page-Jones (1996) y retomada por Richards and Ford (2020) como herramienta de análisis arquitectónico, establece que dos componentes son connascentes si un cambio en uno obliga a cambiar el otro para que el sistema siga siendo correcto. Su valor está en que distingue *tipos* de dependencia y permite ordenarlos por fuerza.

La connascencia **estática** (de nombre, de tipo, de significado, de posición, de algoritmo) es detectable mediante el análisis del código fuente: el compilador, el linter o la revisión de código la hacen visible. La connascencia **dinámica** (de ejecución, de temporización, de valor, de identidad) solo se manifiesta en tiempo de ejecución y es, por tanto, notablemente más difícil de detectar y de corregir (Richards and Ford, 2020). Page-Jones añade tres criterios de evaluación que usaremos a lo largo del análisis: la *fuerza* (es preferible la connascencia estática a la dinámica), la *localidad* (una connascencia fuerte es tolerable si los elementos están próximos, por ejemplo dentro del mismo módulo, y es peligrosa si están separados) y el *grado* (cuántos elementos quedan afectados).

El punto central del análisis que sigue es que **los tres diseños no eliminan la dependencia entre las tres acciones: la trasladan de una forma de connascencia a otra**.

1.2. Orquestación en microservicios

Un servicio mediador —`AuctionCompletionOrchestrator`— conoce el flujo completo e invoca a `Notification`, `Payment` e `Inventory`. El conocimiento del proceso de negocio queda centralizado y explícito, lo que facilita razonar sobre él, instrumentarlo y modificarlo en un solo lugar (Richards and Ford, 2020).

El coste es un acoplamiento dinámico elevado y concentrado en el mediador:

- **Connascencia de ejecución:** el orden importa. Si el mediador invoca la notificación antes que el pago, la corrección del sistema depende de esa secuencia, que no está expresada en ninguna interfaz sino en el cuerpo del mediador.
- **Connascencia de temporización:** si las llamadas son síncronas, los cuatro servicios deben estar disponibles simultáneamente durante la operación. La latencia total es la suma de las latencias y la disponibilidad efectiva es el producto de las disponibilidades individuales, un efecto que Ford et al. (2021) describen como el coste del acoplamiento dinámico síncrono.

- **Grado alto, localidad baja:** la dependencia afecta a cuatro servicios desplegados por separado, es decir, en la peor posición según el criterio de localidad.

1.3. Coreografía en microservicios

En la coreografía no hay mediador: `Auction` publica el evento `AuctionClosed` y cada servicio interesado reacciona. El acoplamiento dinámico disminuye de forma apreciable. Desaparece la connascencia de temporización —el productor no espera al consumidor, que puede estar caído y procesar el mensaje más tarde— y el emisor deja de conocer a sus destinatarios, por lo que añadir un cuarto consumidor no obliga a modificar el emisor (Richards and Ford, 2020).

Sin embargo, la dependencia no desaparece, sino que cambia de naturaleza y de lugar:

- Aparece **connascencia de significado** en torno al contrato del evento: todos los consumidores deben interpretar de la misma manera los campos del mensaje. Es connascencia estática, y por tanto más débil, pero su localidad es pésima, ya que está repartida entre servicios con ciclos de vida independientes.
- El **estado del flujo deja de existir como entidad**. Ningún componente sabe si la operación completa terminó. Richards and Ford (2020) señalan precisamente esto como la desventaja principal de la coreografía: la gestión de errores y la recuperación se vuelven difíciles porque no hay un dueño del proceso.
- Si los consumidores, para completar su trabajo, empiezan a llamarse entre sí, reaparece el acoplamiento dinámico bajo una forma menos visible, lo que Ford et al. (2021) describen como la trampa de la ńcoreografía con comunicación lateralż.

1.4. Servicio de grano grueso en Service-Based Architecture

En SBA las tres acciones viven dentro de un mismo servicio de dominio, sobre una base de datos compartida (Richards and Ford, 2020). La consecuencia desde el punto de vista de la connascencia es la más relevante de toda la pregunta: la dependencia entre las tres acciones pasa a ser **estática** —llamadas a métodos dentro del mismo código base, verificables por el compilador— y su **localidad es máxima**. Según los criterios de Page-Jones, esta es la situación más favorable: una connascencia fuerte dentro de un módulo es aceptable, mientras que una connascencia débil repartida entre servicios remotos no lo es.

No obstante, conviene no sobrevender el estilo. SBA no elimina el acoplamiento, lo relocaliza. La base de datos compartida introduce acoplamiento estático entre servicios a través del esquema, de modo que un cambio en una tabla puede forzar el despliegue coordinado de varios servicios (Richards and Ford, 2020). Es un acoplamiento distinto —estático, detectable en construcción— pero real.

1.5. Fallo del pago después de notificar al ganador

El escenario propuesto por el enunciado es deliberadamente incómodo, porque la notificación es una acción *no transaccional*: una vez enviado el correo, ningún mecanismo técnico lo retira.

En SBA (modelo ACID). Las tres operaciones comparten una única base de datos y, por tanto, un único límite transaccional. Si el pago falla, el *rollback* deshace la actualización del inventario y el sistema queda en un estado consistente: la atomicidad está garantizada por el gestor de base de datos y no por código de compensación escrito a mano (Richards and Ford, 2020). Esta es la ventaja decisiva del estilo en este escenario.

Ahora bien, la garantía ACID cubre únicamente los efectos sobre la base de datos. Si el correo se envió dentro de la transacción y esta se revierte, el ganador ya recibió la notificación de una operación que no ocurrió. La solución correcta, también en SBA, es no ejecutar efectos externos dentro del límite transaccional: se registra la intención de notificar en la misma transacción y un proceso posterior envía el correo solo si la transacción confirmó —el patrón conocido como *transactional outbox* (Ford et al., 2021). El diseño ACID simplifica el problema, pero no exime de ordenar correctamente los efectos secundarios.

En microservicios (modelo BASE). Con una base de datos por servicio no existe un límite transaccional común, y la alternativa clásica —*commit* en dos fases— se descarta por su impacto en disponibilidad y rendimiento (Ford et al., 2021). La consistencia pasa a ser eventual: el sistema acepta un estado intermedio inconsistente y lo corrige mediante una *saga*, es decir, una secuencia de transacciones locales con sus correspondientes transacciones compensatorias (Richards and Ford, 2020).

La compensación funciona bien para el inventario: basta volver a marcar el artículo como disponible. Falla para la notificación, y esa es la cuestión de fondo. Una transacción compensatoria no revierte el efecto, sino que ejecuta una acción que lo neutraliza; cuando el efecto es un correo leído por una persona, lo único disponible es un segundo correo de disculpa. El sistema puede alcanzar consistencia eventual en sus datos y, aun así, quedar en incoherencia semántica frente al usuario. De ahí la regla práctica que se deduce del escenario: **las acciones irreversibles deben ejecutarse lo más tarde posible dentro del flujo**, cuando el resto de pasos ya ha confirmado.

La diferencia entre ambos enfoques es, en el fondo, dónde se paga la complejidad. En SBA la paga el gestor de base de datos; en microservicios la paga el equipo de desarrollo, que debe diseñar compensaciones, garantizar idempotencia en los consumidores y mantener la observabilidad del flujo distribuido (Ford et al., 2021).

1.6. Síntesis

Para el caso concreto de la subasta, el equilibrio favorece al diseño de grano grueso de SBA: las tres acciones forman una unidad de negocio con un requisito de atomicidad real, y separarlas en servicios independientes introduce una complejidad —compensaciones, idempotencia, observabilidad distribuida— que el dominio no pide. Si el contexto obligara a microservicios por otras razones, la orquestación sería preferible a la coreografía en este flujo, porque el estado explícito del mediador es justamente lo que hace manejable la compensación cuando el pago falla (Ford et al., 2021).

	Orquestación (MSA)	**Coreografía (MSA)**	**Grano grueso (SBA)**
Acoplamiento dinámico	Alto y centralizado en el mediador	Bajo entre emisor y consumidores	Inexistente entre las tres acciones
Connascencia dominante	Ejecución y temporización (dinámica)	Significado sobre el contrato del evento (estática, localidad baja)	Nombre y tipo dentro del servicio (estática, localidad alta)
Estado del flujo	Explícito en el mediador	Inexistente	Implícito en la transacción
Consistencia	BASE, saga	BASE, saga	ACID
Precio a pagar	Cuello de botella y punto único de fallo	Ausencia de visibilidad y de gestión de errores	Base de datos compartida como punto de acoplamiento

Cuadro 1: Comparación de los tres enfoques para el cierre de la subasta.

2. Pregunta 2: Diseño para elasticidad extrema y consistencia de datos (venta de entradas para conciertos)

El escenario describe un pico de carga que pasa de cientos a decenas de miles de usuarios concurrentes en segundos. Lo decisivo no es el volumen absoluto, sino la *forma* de la curva: un crecimiento casi instantáneo, impredecible y de corta duración. Esta distinción entre escalabilidad —capacidad de sostener una carga creciente— y elasticidad —capacidad de absorber picos repentinos— es la que orienta toda la respuesta (Richards and Ford, 2020).

2.1. Por qué el cuello de botella es la base de datos

Una Microservices Architecture escala bien la capa de cómputo: cada servicio se replica de forma independiente y se puede dimensionar el que más lo necesita. Pero todas esas instancias terminan convergiendo en almacenes de datos persistentes, y es ahí donde aparece el límite. Con decenas de miles de peticiones por segundo contra un mismo conjunto de asientos, la contención de bloqueos, el número de conexiones disponibles y la latencia de escritura de la base de datos se convierten en el factor limitante (Richards and Ford, 2020). Añadir instancias de servicio no solo no resuelve el problema: lo agrava, porque cada instancia nueva compite por el mismo recurso compartido.

A esto se añade un problema de tiempos. La elasticidad no depende solo de poder arrancar instancias, sino de cuánto tardan en ser útiles. Una instancia de microservicio recién arrancada debe establecer conexiones, calentar cachés y empezar a consultar la base de datos; durante ese intervalo contribuye a la congestión antes de aliviarla. En un pico que se forma en segundos, ese retraso es determinante.

2.2. Por qué Space-Based Architecture resuelve el escenario

La arquitectura basada en espacio se construye sobre una decisión radical: **eliminar la base de datos del camino de la petición** (Richards and Ford, 2020). El modelo,

derivado de la tuple space de Gelernter, sustituye el acceso a disco por memoria replicada entre unidades de procesamiento. Sus componentes son:

- **Processing unit:** contiene la lógica de la aplicación y, junto a ella, la caché de datos en memoria que necesita para operar. Escalar consiste en arrancar más unidades, cada una autosuficiente.
- **Virtualized middleware:** gobierna la topología. Se compone del *messaging grid*, que gestiona la petición de entrada y el reparto de sesiones entre unidades; el *data grid*, responsable de la replicación de datos entre cachés —el componente más importante del estilo—; el *processing grid*, opcional, que coordina peticiones que afectan a varios tipos de unidad; y el *deployment manager*, que arranca y detiene unidades según la carga.
- **Data pumps, data writers y data readers:** el canal asíncrono hacia la persistencia. La unidad de procesamiento publica el cambio en un *data pump* y continúa; un *data writer* lo lleva a la base de datos fuera del camino crítico, y un *data reader* recarga los datos en memoria cuando una unidad arranca o tras una caída.

El efecto sobre la elasticidad es directo. La base de datos deja de ser un recurso en contención y pasa a ser un destino asíncrono; una unidad nueva arranca con los datos ya en memoria y rinde desde el primer momento. Por eso Richards and Ford (2020) clasifican este estilo con las puntuaciones más altas en elasticidad, escalabilidad y rendimiento, y utilizan precisamente la venta de entradas para conciertos como su ejemplo canónico.

El precio es igualmente claro y conviene declararlo: complejidad operativa y de pruebas muy alta, dependencia de productos de middleware especializados, coste elevado y, sobre todo, el riesgo de perder datos que aún no se han escrito en disco si caen varias unidades a la vez.

2.3. Colisiones de datos en cachés replicadas

Una colisión ocurre cuando dos unidades de procesamiento actualizan el mismo dato en sus respectivas cachés dentro de la ventana de tiempo que tarda la replicación en propagarse. Ambas actualizaciones se consideran válidas localmente, pero al replicarse una sobrevive y la otra se pierde silenciosamente. Richards and Ford (2020) identifican cuatro factores que determinan la probabilidad de que esto ocurra:

1. el **número de instancias** que mantienen una réplica de la caché;
2. el **ritmo de actualización** del dato, en actualizaciones por segundo;
3. el **tamaño de la caché**, es decir, la cantidad de registros entre los que se reparten esas actualizaciones;
4. la **latencia de replicación**, que define la duración de la ventana de riesgo.

La venta de asientos numerados es el peor caso posible para los cuatro: muchas unidades activas durante el pico, un ritmo de actualización altísimo, un conjunto de registros reducido —los asientos de un recinto— y una ventana de replicación que, por pequeña que sea, se recorre miles de veces por segundo. Y el dominio no tolera el error: vender dos veces la misma butaca no es una inconsistencia temporal que se corrija sola, es un billete que alguien no podrá usar.

2.4. Mecanismos de mitigación y trade-offs

1. Particionar la caché por propietario único. Es la medida más eficaz. En lugar de replicar todos los asientos en todas las unidades, se asigna cada sección del recinto a una partición y cada partición a una única unidad de procesamiento responsable de sus escrituras. Si solo un componente escribe un dato, la colisión deja de ser posible por construcción: el problema se elimina en vez de gestionarse. El *messaging grid* encamina cada petición a la unidad propietaria de la sección solicitada. El trade-off es que la elasticidad deja de ser uniforme —las secciones más demandadas concentran la carga— y que la caída de una unidad inutiliza temporalmente su sección hasta que otra asume la partición.

2. Reducir la latencia de replicación. Actúa sobre la ventana de riesgo sin cambiar el diseño: red dedicada de baja latencia entre unidades, replicación de deltas en lugar de registros completos y limitación del número de réplicas. Es una mitigación barata, pero solo *reduce* la probabilidad; no la anula. Por sí sola es insuficiente para un dominio que no admite errores.

3. Caché distribuida en lugar de replicada. Los datos residen en un servidor de caché externo con una única fuente de verdad y las unidades los consultan por red. Las colisiones desaparecen porque no hay copias divergentes, a cambio de introducir latencia en cada acceso y un punto central de fallo: se renuncia a parte del rendimiento y de la tolerancia a fallos que justificaban el estilo (Richards and Ford, 2020).

4. Caché híbrida o *near-cache*. Permite combinar lo anterior según la naturaleza del dato, y en este dominio la distinción es natural. El catálogo —conciertos, recintos, mapas de sala, precios— se lee masivamente y casi no cambia: replicado. El estado de ocupación de cada asiento se escribe en competencia: particionado o distribuido. Aplicar la misma política a todos los datos es el error que conviene evitar.

5. Reserva con control de concurrencia y expiración. Complementa a los anteriores dentro de la unidad propietaria. La operación sobre el asiento no es ńescribir ocupadoż, sino ńpasar de libre a reservado si y solo si sigue libreż, una actualización condicional con número de versión que convierte el conflicto en un rechazo explícito y no en una pérdida silenciosa. La reserva se mantiene con un tiempo de expiración que la libera si el pago no se completa.

2.5. Conclusión

Space-Based Architecture es la elección adecuada porque ataca el cuello de botella real del escenario —la base de datos bajo un pico instantáneo— en lugar de escalar la capa que no lo sufre. La consistencia de los asientos numerados no se obtiene del estilo por defecto, sino de una decisión de diseño explícita: particionar el dato crítico de modo que cada asiento tenga un único propietario, y reservar la replicación para los datos de lectura. El trade-off aceptado es conocido y acotado: se sacrifica parte de la elasticidad uniforme y se añade complejidad de encaminamiento a cambio de la garantía de no vender dos veces la misma butaca. En términos de Brewer (2012), es la decisión de priorizar la consistencia en el único punto del sistema donde el negocio no admite otra cosa.

3. Pregunta 3: Análisis de fallos y resiliencia en arquitecturas dirigidas por eventos (topología de broker)

La topología de broker encadena procesadores de eventos sin control central: un evento inicial entra en el sistema y cada procesador que lo consume publica, a su vez, los eventos derivados de su trabajo, a los que reaccionan otros procesadores (Richards and Ford, 2020). El flujo de negocio no está escrito en ninguna parte; emerge de la suma de reacciones locales. Esa propiedad es la fuente de su rendimiento y de su escalabilidad, y también la raíz del problema que plantea el enunciado.

3.1. Impacto del mensaje defectuoso

Un procesador que falla de forma repetida con un mensaje concreto —el caso habitual es un dato mal formado o un campo obligatorio ausente— produce un fallo cualitativamente distinto al de un procesador caído. El procesador funciona; es ese mensaje el que no puede completarse nunca.

Bloqueo de la cola. Si el procesador no confirma el mensaje, el broker lo vuelve a entregar, el procesamiento falla otra vez y el ciclo se repite indefinidamente. En un canal con garantía de orden, el mensaje defectuoso queda en la cabecera y ningún mensaje posterior se entrega: es el *head-of-line blocking*. Un único registro corrupto detiene el procesamiento de todos los mensajes correctos que vienen detrás, y el consumo de recursos del reintento continuo degrada además el rendimiento del procesador.

Interrupción silenciosa de la cadena. Como el procesador nunca termina su trabajo, nunca publica el evento derivado que debía publicar. Todos los procesadores situados aguas abajo quedan esperando un evento que no llegará. En la topología de broker nadie tiene constancia de ello: no existe un componente que conozca el flujo completo ni que mantenga su estado, de modo que la ausencia de un evento es indistinguible de la ausencia de actividad (Richards and Ford, 2020). El sistema no se cae; simplemente deja de hacer una parte de su trabajo.

Experiencia del usuario final. Es el aspecto más grave. La acción inicial del usuario fue aceptada y confirmada —el evento inicial se publicó correctamente—, por lo que la interfaz le devolvió éxito. Las consecuencias de esa acción, sin embargo, se completan solo parcialmente: en un sistema de pedidos, el pedido existe pero nunca se reserva inventario ni se emite la factura. El usuario observa un estado incoherente **sin que el sistema haya mostrado ningún error**, y el fallo suele descubrirse días después, por una reclamación y no por la monitorización.

Consecuencias operativas. Sin manejo central de errores, el diagnóstico exige correlacionar los registros de varios procesadores independientes para reconstruir a posteriori un flujo que nunca estuvo representado en ningún sitio. Si, para desbloquear la cola, alguien descarta el mensaje, el fallo se convierte en pérdida de datos definitiva y silenciosa.

3.2. Solución: Workflow Event Pattern

Richards and Ford (2020) proponen para este caso el *Workflow Event Pattern*, cuyo principio es separar el procesamiento normal de la reparación de errores, de forma que un mensaje defectuoso nunca bloquee el canal principal. El procesador de eventos no intenta resolver el error: detecta que no puede procesar el mensaje, lo delega y sigue trabajando con el siguiente.

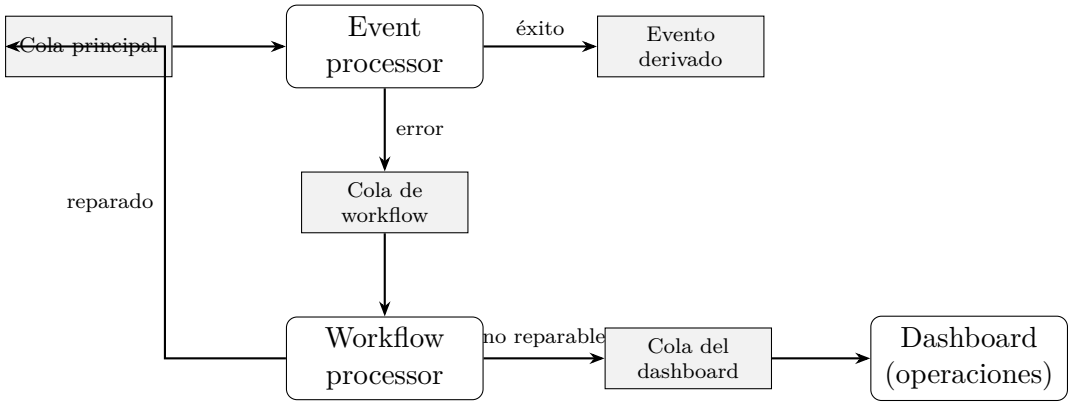


Figura 1: Workflow Event Pattern: el mensaje defectuoso sale del canal principal, se intenta reparar y, si no es posible, acaba en el *dashboard* para intervención manual.

Componentes adicionales.

- **Cola de workflow:** canal dedicado al que el procesador deriva los mensajes que no puede tratar, junto con el contexto del error. Su sola existencia resuelve el bloqueo, porque el mensaje abandona inmediatamente el canal principal.
- **Workflow processor:** componente que recibe el mensaje defectuoso, lo analiza y aplica las reglas de reparación que conoce —completar un campo ausente con un valor por defecto, normalizar un formato de fecha, recuperar un dato de referencia que falta. Si lo consigue, devuelve el mensaje a la cola original como un **mensaje nuevo**.
- **Cola y aplicación de *dashboard*:** destino de los mensajes que el *workflow processor* no sabe reparar. El equipo de operaciones los revisa, los corrige manualmente y los reinyecta. Es el punto donde el fallo deja de ser silencioso y se vuelve visible.

Cómo mitiga el problema. El patrón actúa sobre los tres efectos descritos. Elimina el bloqueo, porque el canal principal nunca retiene un mensaje irreprocesable. Elimina la pérdida silenciosa, porque todo mensaje acaba en uno de dos estados observables: reparado y reprocesado, o en el *dashboard* a la espera de intervención. Y recupera una forma de visibilidad sin introducir un mediador central: el *dashboard* concentra los errores, pero no participa en el flujo normal, de modo que la topología de broker conserva su rendimiento y su desacoplamiento.

3.3. Trade-offs que introduce

- **Procesamiento fuera de orden.** Es el coste explícito del patrón, señalado por el propio enunciado. El mensaje reparado vuelve al final de la cola, después de mensajes

que llegaron más tarde. En dominios donde el orden es semánticamente relevante —actualizaciones sucesivas sobre la misma entidad— esto puede producir un estado final incorrecto, y el sistema debe entonces incorporar números de versión o marcas de tiempo para rechazar actualizaciones obsoletas.

- **Duplicación de conocimiento del dominio.** El *workflow processor* necesita saber qué es un mensaje válido, es decir, replica reglas que ya viven en el procesador original. Aparece connascencia de significado entre dos componentes desplegados por separado (Page-Jones, 1996): un cambio en el contrato obliga a modificar ambos, y si se olvida uno, la reparación empieza a producir mensajes incorrectos en silencio.
- **Complejidad añadida.** Dos canales más, un procesador más y una aplicación de monitorización, con su propio despliegue, sus pruebas y su mantenimiento.
- **Latencia variable.** Un mensaje reparado se completa mucho después que el resto, lo que debe ser aceptable para el negocio.
- **Alcance limitado.** El patrón resuelve errores en los *datos*. No resuelve defectos del procesador: si el fallo es un error de programación, la reparación no aplica y el mensaje terminará igualmente en el *dashboard*, que actúa entonces como red de seguridad y no como solución.

Conviene añadir que el patrón es compatible con las prácticas habituales de mensajería y las complementa: límite de reintentos con espera creciente antes de derivar a la cola de workflow, contador de entregas para detectar mensajes venenosos, consumidores idempotentes para tolerar la reentrega, y conservación del mensaje original junto al reparado para auditoría. La alternativa de fondo —adoptar una topología de mediador, que centraliza el control de errores y el estado del flujo— resuelve el problema de raíz, pero a costa del rendimiento, de la escalabilidad y del desacoplamiento que motivaron la elección del broker (Richards and Ford, 2020); el Workflow Event Pattern es precisamente la forma de conservar esas ventajas pagando solo por la resiliencia que falta.

4. Pregunta 4: El dilema de la granularidad y el acoplamiento en microservicios (sistema de recursos humanos)

El equipo obtuvo el peor resultado posible: pagó íntegro el coste de la distribución —red, serialización, despliegues independientes, observabilidad— y no recibió ninguna de sus ventajas. El sistema resultante es más frágil que el monolito del que partió, porque cada llamada interna que antes era una invocación en memoria es ahora una llamada remota que puede fallar, tardar o perderse (Richards and Ford, 2020). Este resultado tiene un nombre: **monolito distribuido**.

4.1. El error fundamental: confundir entidad con bounded context

El equipo tomó como eje de descomposición el esquema de datos del monolito. Allí donde había una tabla —Empleado, Departamento, Nómina, Beneficios— crearon un

servicio. Ford et al. (2021) denominan a esto la *trampa de la entidad* (*entity trap*) y la describen como uno de los errores más frecuentes al definir límites de servicio.

El problema es que una entidad de datos no es una capacidad de negocio. El *bounded context* de Evans (2003), que Richards and Ford (2020) sitúan como el fundamento conceptual de los microservicios, no delimita un conjunto de tablas sino un ámbito en el que un modelo del dominio y su lenguaje tienen un significado coherente y único. Contratar a un empleado no es una operación sobre la tabla *Empleado*: es un proceso de negocio que crea un empleado, lo adscribe a un departamento, lo da de alta en nómina y le asigna beneficios. Al partir por entidades, el equipo trazó los límites justo por el medio de ese proceso.

De ahí se siguen dos consecuencias encadenadas:

1. **Toda operación de negocio se convierte en una transacción distribuida.** Lo que en el monolito era una transacción ACID sobre cuatro tablas pasa a requerir cuatro transacciones locales en cuatro bases de datos, sin atomicidad posible entre ellas. La corrección pasa a depender de sagas y compensaciones que el equipo no había previsto (Ford et al., 2021).

2. **La coreografía síncrona reintroduce el acoplamiento dinámico.** Los servicios se llaman unos a otros en cadena para completar una sola operación. La disponibilidad del conjunto es el producto de las disponibilidades individuales y la latencia es la suma de las latencias: el sistema es menos disponible y más lento que el monolito original.

Conviene nombrar también el supuesto implícito que llevó allí: la idea de que ńcuanto más pequeño el servicio, mejorż. Ford et al. (2021) insisten en que la granularidad no es un valor a maximizar sino un trade-off a equilibrar, y en que modularidad y granularidad son cosas distintas: la primera indica cómo se separan las responsabilidades, la segunda cuántas asume cada servicio desplegable.

4.2. Nueva estrategia de descomposición

La descomposición no debe partir del esquema de datos, sino de las capacidades de negocio, y debe someterse después a las fuerzas que justifican separar o unir servicios. El enunciado pide aplicar tres de ellas.

Propósito. Cada servicio debe corresponder a una capacidad de dominio con cohesión funcional propia: debe poder describirse con un verbo de negocio, no con un sustantivo del modelo de datos (Ford et al., 2021). ńGestionar la vida laboral del empleadoż es un propósito; ńadministrar la tabla Departamentoż no lo es. Esta es la fuerza que decide la partición inicial.

Transacciones. Lo que el negocio exige que ocurra de forma atómica debe residir dentro de un único servicio, con una única base de datos y una única transacción ACID. Es el criterio más fuerte para *unir* servicios: si dos operaciones no pueden quedar en estados distintos sin dejar el sistema incorrecto, separarlas obliga a una saga, y una saga es un coste permanente de diseño, de pruebas y de operación (Ford et al., 2021). En el caso de recursos humanos, crear al empleado y adscribirlo a su departamento es exactamente esa clase de par indivisible.

Coreografía. La intensidad de la comunicación entre servicios es un indicador de que el límite está mal trazado. Si dos servicios necesitan hablar constantemente para completar operaciones ordinarias, el dominio está diciendo que pertenecen al mismo ámbito y deben fusionarse (Ford et al., 2021). Como regla práctica: una operación de negocio habitual no debería requerir más de una o dos interacciones entre servicios.

A estas tres se añaden, como fuerzas de separación, la volatilidad del código, las necesidades diferenciadas de escalado y los requisitos de seguridad; esta última es relevante aquí, porque los datos salariales exigen un tratamiento más restrictivo que el organigrama y justifican por sí solos un límite de servicio (Ford et al., 2021).

4.3. Propuesta concreta



Figura 2: La descomposición por entidades obliga a una coreografía síncrona entre casi todos los servicios; la descomposición por capacidad reduce la contratación a una llamada y un evento.

`EmployeeManagement`
: Reúne al empleado, su adscripción a un departamento y la estructura organizativa. Justificación: las tres cosas cambian juntas en los procesos de alta, traslado y baja, y deben hacerlo de forma atómica (criterio de transacciones); su propósito común es ńmantener la situación laboral del empleadoż.

`PayrollAndBenefits`
: Reúne nómina y beneficios. Justificación: los beneficios son, en su mayoría, deducciones y aportaciones que intervienen en el cálculo de la nómina, de modo que separarlos generaba una conversación constante entre ambos servicios (criterio de coreografía) y una relación de datos estrecha. Además comparten un requisito de seguridad más estricto que el resto del sistema.

`Recruiting`
: Opcional según el alcance del sistema: gestiona vacantes y candidatos. Su propósito es distinto —termina donde empieza la contratación—, su volatilidad es mayor y su carga es independiente, de modo que las fuerzas de separación lo justifican.

4.4. La operación de ńcontrataciónż

Con esta descomposición, la contratación deja de ser una coreografía de cuatro servicios:

1. El cliente realiza **una única llamada síncrona** a `EmployeeManagement`. El servicio crea el empleado y su adscripción al departamento dentro de una **transacción ACID**

local, porque ambos datos viven en su base de datos. Si algo falla, la transacción se revierte y no queda ningún estado intermedio que compensar.

2. Confirmada la transacción, `EmployeeManagement` publica el evento `EmployeeHired` con los datos necesarios para el resto del proceso.

3. `PayrollAndBenefits` consume el evento de forma **asíncrona** y realiza su propio trabajo —alta en nómina y asignación de beneficios por defecto— en una segunda transacción local.

La consistencia entre ambos servicios es eventual, y el dominio lo admite sin dificultad: el empleado debe figurar en la nómina antes del cierre del período, no en el mismo milisegundo en que se firma el contrato. Esta es la comprobación que el equipo original no hizo: antes de aceptar consistencia eventual hay que verificar que el negocio la tolera, y antes de separar hay que verificar que no la tolera.

El usuario obtiene respuesta inmediata en cuanto la transacción crítica confirma; la caída temporal de `PayrollAndBenefits` no impide contratar, porque el evento permanece en la cola hasta que el servicio se recupera. Se ha pasado de tres o cuatro llamadas síncronas encadenadas —con su acoplamiento dinámico y su disponibilidad multiplicativa— a una llamada síncrona y un evento asíncrono, que es lo que Ford et al. (2021) describen como el objetivo de un reparto de límites correcto: que las transacciones queden dentro de los servicios y la comunicación entre ellos sea asíncrona y escasa.

5. Pregunta 5: Comparación de atributos de calidad y trade-offs (sistema de transacciones financieras)

El CTO plantea tres requisitos que no apuntan en la misma dirección: consistencia absoluta, agilidad de despliegue y viabilidad con un equipo de experiencia mixta. La decisión arquitectónica consiste precisamente en identificar cuál de ellos es innegociable y cuáles admiten una solución parcial. En un sistema de transacciones financieras, el primero lo es: un asiento contable que quede a medias no es una inconsistencia temporal, es dinero mal contabilizado.

5.1. Requisito 1: consistencia de datos absoluta (ACID)

Service-Based Architecture. Los servicios de dominio comparten una base de datos, de modo que existe un único límite transaccional y la atomicidad la garantiza el gestor de base de datos (Richards and Ford, 2020). Un adeudo y su correspondiente abono se escriben dentro de la misma transacción: o constan ambos o no consta ninguno, sin código adicional. Las cuatro propiedades ACID se obtienen del motor, no del diseño de la aplicación.

Microservices Architecture. El estilo exige que cada servicio posea su propia base de datos; esa es la condición que hace posible el despliegue verdaderamente independiente (Richards and Ford, 2020). En consecuencia, una operación que afecte a dos servicios no puede ser atómica. La alternativa pasa por una *saga*: transacciones locales encadenadas

con compensaciones, bajo un modelo BASE de consistencia eventual (Ford et al., 2021). Esto significa que existe un intervalo —por breve que sea— en el que el adeudo está registrado y el abono no.

Veredicto. Service-Based Architecture cumple el requisito; Microservices Architecture no. La consistencia eventual no es una versión más lenta de la consistencia fuerte: es una garantía distinta, que admite estados intermedios visibles. Se puede construir un sistema financiero correcto sobre sagas, pero entonces el requisito deja de ser ńconsistencia absolutaż y pasa a ser ńconsistencia eventual con compensaciones auditadasż, que es otra cosa. La objeción habitual —hacer un microservicio lo bastante grande como para contener la transacción— es en realidad un argumento a favor de SBA: un servicio de grano grueso con base de datos compartida es, por definición, el estilo que estamos comparando.

5.2. Requisito 2: despliegue independiente

Aquí conviene deshacer un malentendido frecuente: SBA no es un monolito. Sus servicios de dominio se despliegan de forma independiente, y esa es exactamente la propiedad que lo distingue de una arquitectura monolítica en capas (Richards and Ford, 2020). Un cambio en el servicio de informes no obliga a desplegar el de liquidación.

El matiz está en la base de datos compartida, que introduce un punto de acoplamiento estático: un cambio de esquema puede afectar a varios servicios y forzar un despliegue coordinado. La independencia es, por tanto, real pero condicionada —independiente ńcon asteriscoż—, y el asterisco se activa solo en los cambios de esquema, no en los cambios de lógica, que son la mayoría.

En microservicios la independencia es completa, porque cada servicio posee sus datos y nada obliga a coordinar. El coste está en otra parte: ese grado de autonomía requiere canalizaciones de integración continua maduras, automatización de pruebas de contrato, versionado de interfaces, observabilidad distribuida y una cultura operativa que lo sostenga (Richards and Ford, 2020). La independencia de despliegue no es gratuita; se paga en infraestructura y en disciplina de equipo.

El acoplamiento del esquema, además, es mitigable con técnicas conocidas: asignar la propiedad de cada conjunto de tablas a un único servicio —los demás acceden a través de él—, usar esquemas lógicos separados dentro de la misma base de datos (el caso de base de datos ńparcialmente compartidaż que contempla el propio estilo) y aplicar migraciones en dos fases de tipo *expand/contract*, que permiten desplegar los servicios en cualquier orden.

5.3. Requisito 3: equipo de experiencia mixta

Este requisito, que a primera vista parece el más blando, es el que en la práctica decide, y refuerza la misma conclusión.

Microservices Architecture es el estilo con menor puntuación en simplicidad y mayor coste de todos los analizados por Richards and Ford (2020). Exige dominar contenedores, orquestación, trazas distribuidas, patrones de resiliencia y, sobre todo, sagas. Y una saga mal implementada en un sistema financiero no produce un fallo visible: produce una compensación que no se ejecuta, un mensaje duplicado que contabiliza dos veces o un estado intermedio que alguien consulta. Son errores silenciosos, difíciles de reproducir y con

consecuencias contables. Pedir a un equipo de experiencia mixta que escriba correctamente compensaciones distribuidas en el dominio financiero es, sencillamente, un riesgo mal calibrado.

SBA, en cambio, es de los estilos distribuidos el más próximo a lo que un equipo con experiencia desigual ya sabe hacer: pocos servicios de grano grueso, transacciones de base de datos convencionales, despliegue por servicio. Mantiene la modularidad y la testabilidad sin exigir la madurez operativa de los microservicios.

5.4. Síntesis y trade-offs que el CTO acepta

Requisito	**Service-Based**	**Microservices**
1. Consistencia ACID	Garantizada por la base de datos compartida	No alcanzable; sagas y consistencia eventual
2. Despliegue independiente	Sí, condicionado por el esquema compartido	Sí, completo, con alto coste de infraestructura
3. Equipo de experiencia mixta	Complejidad operativa moderada	Complejidad y riesgo elevados
Resultado	**Cumple los tres**	Cumple el segundo, incumple el primero

Cuadro 2: Evaluación de ambos estilos frente a los tres requisitos del CTO.

La recomendación es **Service-Based Architecture**, y los trade-offs que el CTO asume al priorizar consistencia sobre agilidad son los siguientes:

- **Coordinación en los cambios de esquema.** Las migraciones que afecten a tablas compartidas requerirán planificación conjunta. Es un coste recurrente y acotado, mitigable con *expand/contract*.
- **Escalado de grano grueso.** No se podrá escalar una funcionalidad concreta de forma aislada, sino el servicio completo que la contiene; y la base de datos compartida fija un techo de escalabilidad que los microservicios no tienen.
- **Radio de impacto mayor.** Un defecto en un servicio de grano grueso afecta a más funcionalidad que uno en un microservicio, y la base de datos compartida constituye un punto único de fallo. El sistema forma un solo *quantum* arquitectónico, de modo que todos los servicios comparten las mismas características operativas y no es posible dar a uno de ellos una disponibilidad o una elasticidad distintas (Richards and Ford, 2020).
- **Menor agilidad en la evolución.** Entre dos cambios equivalentes, el ciclo será algo más lento que en una arquitectura de microservicios bien operada.

Conviene cerrar señalando que la decisión no es irreversible. No todo el sistema tiene el mismo requisito de consistencia: la notificación al cliente, la generación de informes o la detección de fraude toleran consistencia eventual y pueden extraerse más adelante como servicios con sus propios datos, dejando el núcleo transaccional en SBA. Esta evolución selectiva es lo que Ford et al. (2017) describen como arquitectura evolutiva, y recomiendan sostenerla con *fitness functions* automatizadas —por ejemplo, una verificación en la

canalización de integración que impida que un servicio acceda a tablas de las que no es propietario— para que el acoplamiento por la base de datos compartida no se degrade con el tiempo hasta convertir el sistema en el monolito que se quería evitar.

6. Pregunta 6: El legado de SOA y su influencia en arquitecturas modernas

SOA se construyó sobre una premisa razonable: si una capacidad de negocio ya está implementada, ninguna otra parte de la empresa debería volver a implementarla. De ahí su taxonomía de servicios —de negocio, empresariales, de aplicación y de infraestructura— y el motor de orquestación encargado de componerlos (Richards and Ford, 2020). La reutilización no era un efecto secundario del estilo: era su objetivo de primer orden.

El resultado histórico es conocido y conviene enunciar con precisión por qué ocurrió, porque es lo que explica la reacción de los microservicios.

6.1. La reutilización como fuente de acoplamiento

Un servicio empresarial de ńvalidación de clientesż utilizado por treinta aplicaciones distintas acumula los requisitos de las treinta. Cada consumidor necesita un matiz propio, y el servicio crece hasta volverse difícil de entender; al mismo tiempo, deja de poder cambiar, porque cualquier modificación afecta a todos sus consumidores y exige pruebas y coordinación a escala de empresa (Richards and Ford, 2020). La reutilización máxima produce, así, acoplamiento máximo: el servicio compartido se convierte simultáneamente en un cuello de botella técnico y organizativo.

El motor de orquestación agravó el efecto. Al concentrar en un punto central la composición de los servicios, terminó albergando lógica de negocio que no pertenecía a ningún servicio concreto, y se convirtió en un punto único de fallo y en un cuello de botella de rendimiento y de entrega: cualquier cambio en un flujo pasaba por el mismo componente y por el mismo equipo (Richards and Ford, 2020).

Debe señalarse, sin embargo, que la orquestación es solo una de las variantes de SOA, no su definición. SOA admite igualmente la coreografía, en la que los servicios se comunican sin motor central; la crítica aquí formulada se dirige a la variante orquestada que se impuso en la industria, no al estilo en su conjunto.

6.2. La inversión del principio en microservicios

Microservices Architecture no mejora la reutilización de SOA: **invierte el criterio**. Si la reutilización genera acoplamiento y el objetivo prioritario del estilo es el desacoplamiento y el despliegue independiente, entonces la reutilización deja de ser un fin. Richards and Ford (2020) lo formulan de manera explícita: en microservicios se prefiere la duplicación al acoplamiento.

Lo decisivo es que esa inversión no se aplica por igual a todo. El estilo distingue dos clases de reutilización y las trata de forma opuesta.

Reutilización de lógica de negocio: se evita. La validación de clientes se implementa dentro de cada *bounded context* que la necesita, y cada una valida lo que su contexto entiende por ńcliente válidoż (Evans, 2003). Si dos contextos necesitan los mismos datos, se

replican mediante eventos en lugar de consultarse mutuamente, de modo que cada servicio conserva su autonomía. Se acepta deliberadamente escribir dos veces un código parecido a cambio de que cada servicio pueda desplegarse, escalar y evolucionar sin consultar a nadie.

Esto no debe leerse como un rechazo dogmático a compartir código. Ford et al. (2021) tratan la reutilización como un análisis de trade-offs: cuando la lógica es muy volátil y compartirla obligaría a desplegar todos los consumidores a la vez, la duplicación gana; cuando es estable y su duplicación sería costosa o peligrosa —un cálculo fiscal o un algoritmo normativo—, una biblioteca versionada, o incluso un servicio compartido, puede ser la opción correcta. La regla es que compartir debe justificarse, no presuponerse.

Reutilización operativa: se centraliza. El registro de trazas, la monitorización, la autenticación, el descubrimiento de servicios o los cortacircuitos no son lógica de dominio: son preocupaciones transversales idénticas en todos los servicios, y duplicarlas no aporta autonomía alguna, solo inconsistencia. Para ellas, microservicios emplea el patrón **sidecar**: un componente que se despliega junto a cada servicio y asume esas funciones, de modo que el conjunto de sidecars forma una malla de servicios (*service mesh*) gobernada de manera unificada (Richards and Ford, 2020).

Aquí reside la asimetría que responde a la pregunta: **microservicios acepta el acoplamiento donde no afecta al dominio y lo rechaza donde sí**. El sidecar crea un acoplamiento operativo real —todos los servicios dependen de él—, pero ese acoplamiento no impide que un equipo cambie sus reglas de negocio y despliegue sin coordinarse con los demás. El servicio empresarial compartido de SOA sí lo impedía.

6.3. Cuándo sigue siendo apropiada la orquestación

La orquestación no quedó invalidada; lo que quedó invalidado fue aplicarla como mecanismo universal de composición a escala de empresa. Un mediador local —un servicio que posee un flujo concreto y coordina a los demás— es la opción correcta cuando concurren varias de estas condiciones (Ford et al., 2021):

- el flujo es **largo** y atraviesa muchos pasos con **ramificación condicional**;
- mantiene **estado durante días o semanas**, incluso con intervención humana;
- el negocio necesita **saber en qué punto está** cada instancia del proceso;
- los errores exigen **compensación estricta y ordenada**, no simple reintento;
- existe una **obligación de auditoría** sobre qué se hizo, cuándo y por qué.

Ejemplo: solicitud de hipoteca. El proceso abarca la solicitud, la verificación de ingresos, la consulta de historial crediticio, la tasación del inmueble —que depende de un tercero y puede tardar días—, la decisión de riesgo, la formalización ante notario y el desembolso. Es un flujo que dura semanas, se ramifica según el resultado de cada paso, intercala tareas humanas, debe poder suspenderse y reanudarse, y exige dejar constancia auditable de cada decisión. Además, si la operación se cancela después de la tasación, hay que ejecutar compensaciones en un orden determinado: liberar la reserva de fondos, cancelar la póliza de seguro asociada y notificar a las partes.

Resolver esto por coreografía sería un error de diseño. El estado del proceso no residiría en ningún sitio, responder a la pregunta ń£por qué se denegó esta hipoteca?ż obligaría a reconstruir el historial a partir de los registros de siete servicios, y la compensación ordenada sería prácticamente inviable. Un servicio orquestador de solicitudes, que posee la máquina de estados del proceso, convierte todo eso en explícito y consultable.

El trade-off se acepta con los ojos abiertos: el orquestador introduce acoplamiento dinámico con los servicios que coordina, concentra conocimiento del dominio y es un punto de fallo. La diferencia esencial con el motor de orquestación de SOA es el **alcance**: aquel era único, central y responsable de todos los flujos de la empresa; este posee un solo flujo de negocio, pertenece a un solo equipo y se despliega como un servicio más. Esa es, en definitiva, la lección que los microservicios extrajeron de SOA: el problema nunca fue la orquestación, sino su centralización a escala empresarial y la reutilización convertida en obligación.

Referencias

Brewer, E. (2012). CAP Twelve Years Later: How the ńRulesż Have Changed. *Computer*, 45(2):23–29.

Evans, E. (2003). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley, Boston, MA.

Ford, N., Parsons, R., and Kua, P. (2017). *Building Evolutionary Architectures: Automated Software Governance*. O'Reilly Media, Sebastopol, CA.

Ford, N., Richards, M., Sadalage, P., and Dehghani, Z. (2021). *Software Architecture: The Hard Parts. Modern Trade-Off Analyses for Distributed Architectures*. O'Reilly Media, Sebastopol, CA.

Page-Jones, M. (1996). *What Every Programmer Should Know About Object-Oriented Design*. Dorset House, New York, NY.

Richards, M. and Ford, N. (2020). *Fundamentals of Software Architecture: An Engineering Approach*. O'Reilly Media, Sebastopol, CA.

Declaración de Uso de la Inteligencia Artificial

Para la elaboración de este seminario, el equipo utilizó la herramienta Claude (versión Claude Opus 5, a través de Claude Code). Se empleó con tres propósitos: la lectura y el resumen comparado de los capítulos de la bibliografía recomendada, la elaboración de borradores de las respuestas a partir de los esquemas de análisis discutidos por el equipo, y la generación del formato LaTeX del documento, incluidas las tablas comparativas y los diagramas en TikZ.

El resultado se integró de la siguiente forma: los borradores generados se usaron como base de cada sección y fueron posteriormente revisados, contrastados con las fuentes citadas y reescritos por el integrante responsable de cada pregunta, quien verificó que cada afirmación atribuida a la bibliografía se corresponda con el contenido real de los capítulos indicados. Las tablas de trade-offs y los diagramas fueron comprobados y ajustados manualmente.

Todos los integrantes del equipo hicieron uso de la herramienta en la pregunta de la que fueron responsables. Todo el contenido generado fue revisado, verificado y adaptado por el equipo, quien asume la responsabilidad final del documento.